

AI Assisted Coding

Lab Assignment - 16.1

Name : P. Gouri Prasad Varma

Hall.t.No. : 2303A51035

Batch No. : 01

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Lab Objectives:

- Understand schema design principles for relational databases.
- Use AI-assisted tools to generate SQL schema definitions.
- Write and refine SQL queries (CRUD + aggregate operations).
- Explore how AI can optimize queries and ensure normalization.

Task 1 – Student Information System Schema

Task: Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
 - o Insert a new student record.
 - o Fetch all courses enrolled by a student.
 - o Count number of students in each course.

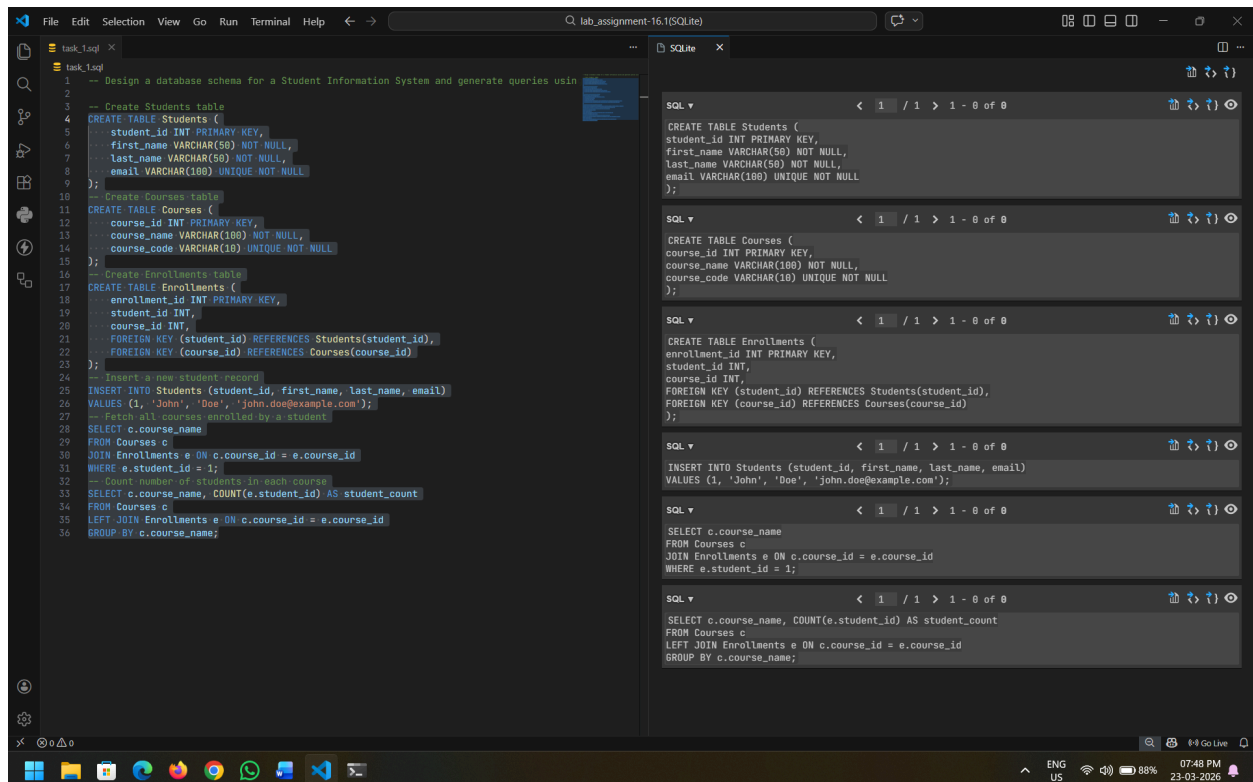
Expected Output:

- SQL CREATE TABLE statements and queries for student– course relationships.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_1.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_1.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:



Task 2 – Hospital Management Database

Task: Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
 - List all appointments for a specific doctor.
 - Retrieve patient history by patient ID.
 - Count total patients treated by each doctor.

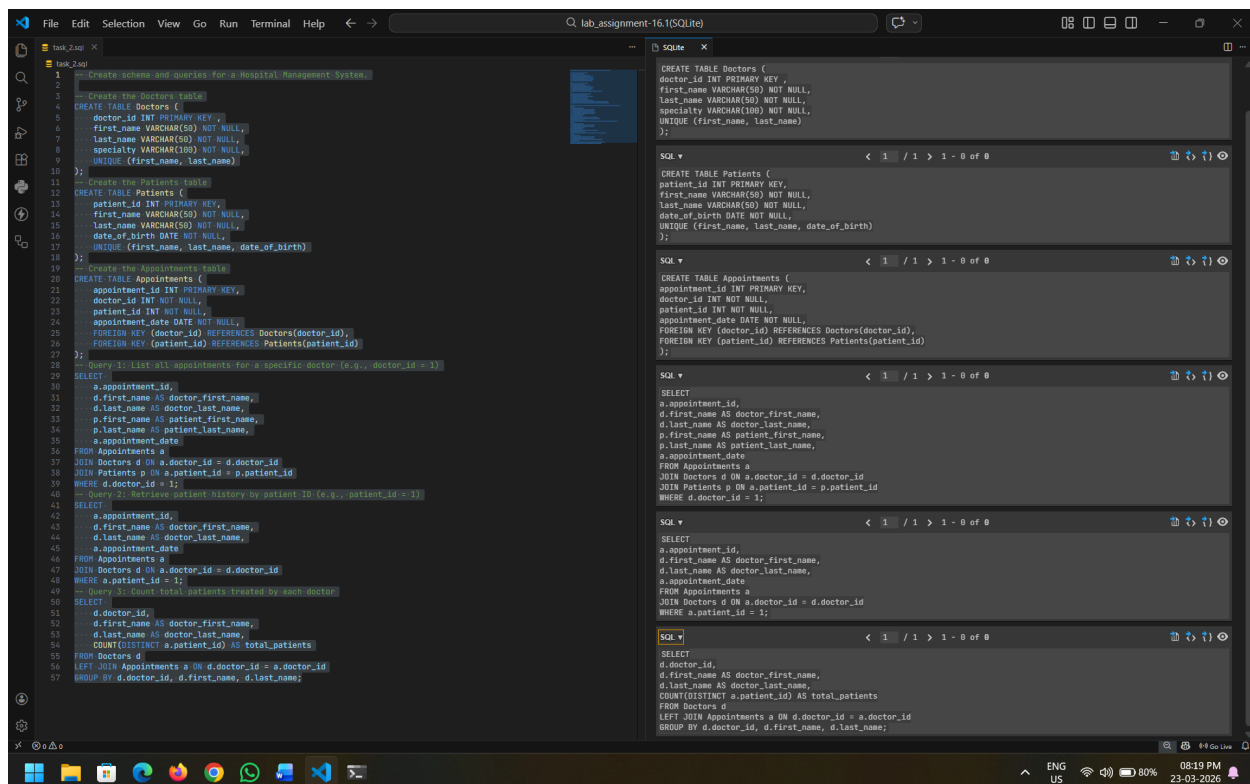
Expected Output:

- Normalized schema and SQL queries with joins.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_2.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_2.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:



```
1 -- Create schema and queries for a Hospital Management System.
2
3 -- Create the Doctors Table
4 CREATE TABLE Doctors (
5     doctor_id INT PRIMARY KEY,
6     first_name VARCHAR(50) NOT NULL,
7     last_name VARCHAR(50) NOT NULL,
8     specialty VARCHAR(100) NOT NULL,
9     UNIQUE (first_name, last_name)
10 );
11
12 -- Create the Patients Table
13 CREATE TABLE Patients (
14     patient_id INT PRIMARY KEY,
15     first_name VARCHAR(50) NOT NULL,
16     last_name VARCHAR(50) NOT NULL,
17     date_of_birth DATE NOT NULL,
18     UNIQUE (first_name, last_name, date_of_birth)
19 );
20
21 -- Create the Appointments Table
22 CREATE TABLE Appointments (
23     appointment_id INT PRIMARY KEY,
24     doctor_id INT NOT NULL,
25     patient_id INT NOT NULL,
26     appointment_date DATE NOT NULL,
27     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
28     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
29 );
30
31 -- Query 1: List all appointments for a specific doctor (e.g., doctor_id = 1)
32 SELECT
33     a.appointment_id,
34     d.first_name AS doctor_first_name,
35     d.last_name AS doctor_last_name,
36     p.first_name AS patient_first_name,
37     p.last_name AS patient_last_name,
38     a.appointment_date
39 FROM Appointments a
40 JOIN Doctors d ON a.doctor_id = d.doctor_id
41 JOIN Patients p ON a.patient_id = p.patient_id
42 WHERE d.doctor_id = 1;
43
44 -- Query 2: Retrieve patient history by patient ID (e.g., patient_id = 1)
45 SELECT
46     a.appointment_id,
47     d.first_name AS doctor_first_name,
48     d.last_name AS doctor_last_name,
49     a.appointment_date
50 FROM Appointments a
51 JOIN Doctors d ON a.doctor_id = d.doctor_id
52 WHERE a.patient_id = 1;
53
54 -- Query 3: Count total patients treated by each doctor
55 SELECT
56     d.doctor_id,
57     d.first_name AS doctor_first_name,
58     d.last_name AS doctor_last_name,
59     COUNT(DISTINCT a.patient_id) AS total_patients
60 FROM Doctors d
61 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
62 GROUP BY d.doctor_id, d.first_name, d.last_name;
```

```
CREATE TABLE Doctors (
  doctor_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  specialty VARCHAR(100) NOT NULL,
  UNIQUE (first_name, last_name)
);

CREATE TABLE Patients (
  patient_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  date_of_birth DATE NOT NULL,
  UNIQUE (first_name, last_name, date_of_birth)
);

CREATE TABLE Appointments (
  appointment_id INT PRIMARY KEY,
  doctor_id INT NOT NULL,
  patient_id INT NOT NULL,
  appointment_date DATE NOT NULL,
  FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
  FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

SELECT
  a.appointment_id,
  d.first_name AS doctor_first_name,
  d.last_name AS doctor_last_name,
  p.first_name AS patient_first_name,
  p.last_name AS patient_last_name,
  a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE d.doctor_id = 1;

SELECT
  a.appointment_id,
  d.first_name AS doctor_first_name,
  d.last_name AS doctor_last_name,
  a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 1;

SELECT
  d.doctor_id,
  d.first_name AS doctor_first_name,
  d.last_name AS doctor_last_name,
  COUNT(DISTINCT a.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.first_name, d.last_name;
```

Task 3 – Library Database

Task: Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:

- o Retrieve all books currently issued.
- o Find overdue books (loan date > 30 days).
- o Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_3.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_3.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:

The screenshot shows a code editor with a schema design for a library management system and several SQL queries. The schema includes tables for Books, Members, and Loans, with their respective columns and data types. The queries demonstrate joins and conditions for retrieving data from the database.

```
-- Design schema for a Library Management System
-- Schema Design for Library Management System
CREATE TABLE Books (
  BookID INT PRIMARY KEY,
  Title VARCHAR(255) NOT NULL,
  Author VARCHAR(255) NOT NULL,
  PublishedDate DATE,
  ISBN VARCHAR(20) UNIQUE
);

CREATE TABLE Members (
  MemberID INT PRIMARY KEY,
  FirstName VARCHAR(255) NOT NULL,
  LastName VARCHAR(255) NOT NULL,
  MembershipDate DATE
);

CREATE TABLE Loans (
  LoanID INT PRIMARY KEY,
  BookID INT,
  MemberID INT,
  LoanDate DATE,
  ReturnDate DATE,
  FOREIGN KEY (BookID) REFERENCES Books(BookID),
  FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);

-- Indexing Strategy
-- To optimize query performance, we can create indexes on the following columns:
-- 1. Books(Title) - for faster search by book title.
-- 2. Loans(LoanDate) - for efficient retrieval of overdue books.
CREATE INDEX idx_books_title ON Books(Title);
CREATE INDEX idx_loans_loan_date ON Loans(LoanDate);

-- SQL Queries
-- 1. Retrieve all books currently issued (i.e., not returned yet).
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;

-- 2. Find overdue books (loan date > 30 days).
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND l.LoanDate < DATE_SUB(CURDATE(), INTERVAL 30 DAY);

-- 3. Count number of books loaned by each member.
SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Task 4 – Real-Time Application: Online Shopping Database

Scenario: Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
 - o Retrieve all orders by a user.
 - o Find the most purchased product.
 - o Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_4.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_4.db r/w
sqlite>
```

The screenshot displays a SQL development environment with two panes. The left pane, titled 'lab_assignment-16.1(SQLite)', contains a series of SQL queries for creating tables, inserting data, and performing calculations. The right pane, titled 'lab', shows the execution results of these queries, including table creation messages and the output of a complex query involving joins and aggregations.

Left Pane (lab_assignment-16.1(SQLite)):

```
CREATE TABLE Suppliers (
  SupplierID INT PRIMARY KEY,
  SupplierName VARCHAR(100) NOT NULL,
  Address VARCHAR(200) NOT NULL,
  City VARCHAR(50) NOT NULL,
  Country VARCHAR(50) NOT NULL,
  ContactName VARCHAR(50) NOT NULL,
  ContactTitle VARCHAR(50) NOT NULL,
  CreateDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL,
  Price DECIMAL(10, 2) NOT NULL,
  Stock INT NOT NULL,
  CreateDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  SupplierID INT FOREIGN KEY REFERENCES Suppliers(SupplierID),
  ProductID INT FOREIGN KEY REFERENCES Products(ProductID),
  OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  TotalAmount DECIMAL(10, 2) NOT NULL,
  Status VARCHAR(20) DEFAULT 'PENDING'
);

CREATE TABLE OrderDetails (
  OrderID INT,
  ProductID INT,
  Quantity INT NOT NULL,
  Price DECIMAL(10, 2) NOT NULL,
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

-- Insert data into Suppliers
INSERT INTO Suppliers (SupplierID, SupplierName, Address, City, Country, ContactName, ContactTitle)
VALUES (1, 'Supplier A', '123 Main St', 'New York', 'USA', 'John Doe', 'Manager'),
(2, 'Supplier B', '456 Elm St', 'Los Angeles', 'USA', 'Jane Smith', 'Manager'),
(3, 'Supplier C', '789 Oak St', 'Chicago', 'USA', 'Mike Johnson', 'Manager'),
(4, 'Supplier D', '101 Pine St', 'Houston', 'USA', 'Sarah Brown', 'Manager'),
(5, 'Supplier E', '202 Cedar St', 'Phoenix', 'USA', 'David Wilson', 'Manager');

-- Insert data into Products
INSERT INTO Products (ProductID, ProductName, Price, Stock)
VALUES (1, 'Product A', 10.00, 100),
(2, 'Product B', 20.00, 50),
(3, 'Product C', 15.00, 75),
(4, 'Product D', 30.00, 30),
(5, 'Product E', 25.00, 60);

-- Insert data into Orders
INSERT INTO Orders (OrderID, SupplierID, ProductID, OrderDate, TotalAmount, Status)
VALUES (1, 1, 1, '2023-01-01', 10.00, 'PENDING'),
(2, 2, 2, '2023-01-05', 20.00, 'PENDING'),
(3, 3, 3, '2023-01-10', 15.00, 'PENDING'),
(4, 4, 4, '2023-01-15', 30.00, 'PENDING'),
(5, 5, 5, '2023-01-20', 25.00, 'PENDING');

-- Insert data into OrderDetails
INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price)
VALUES (1, 1, 10, 10.00),
(2, 2, 5, 20.00),
(3, 3, 7, 15.00),
(4, 4, 3, 30.00),
(5, 5, 6, 25.00);

-- Query to calculate total revenue by supplier
SELECT s.SupplierID, s.SupplierName, SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Suppliers s
JOIN Orders o ON s.SupplierID = o.SupplierID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY s.SupplierID, s.SupplierName;

-- Query to calculate total revenue by product
SELECT p.ProductID, p.ProductName, SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Products p
JOIN Orders o ON p.ProductID = o.ProductID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY p.ProductID, p.ProductName;

-- Query to calculate total revenue by category
SELECT c.CategoryID, c.CategoryName, SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Categories c
JOIN Orders o ON c.CategoryID = o.CategoryID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY c.CategoryID, c.CategoryName;
```

Right Pane (lab):

```
CREATE TABLE Suppliers (
  SupplierID INT PRIMARY KEY,
  SupplierName VARCHAR(100) NOT NULL,
  Address VARCHAR(200) NOT NULL,
  City VARCHAR(50) NOT NULL,
  Country VARCHAR(50) NOT NULL,
  ContactName VARCHAR(50) NOT NULL,
  ContactTitle VARCHAR(50) NOT NULL,
  CreateDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL,
  Price DECIMAL(10, 2) NOT NULL,
  Stock INT NOT NULL,
  CreateDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  SupplierID INT FOREIGN KEY REFERENCES Suppliers(SupplierID),
  ProductID INT FOREIGN KEY REFERENCES Products(ProductID),
  OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  TotalAmount DECIMAL(10, 2) NOT NULL,
  Status VARCHAR(20) DEFAULT 'PENDING'
);

CREATE TABLE OrderDetails (
  OrderID INT,
  ProductID INT,
  Quantity INT NOT NULL,
  Price DECIMAL(10, 2) NOT NULL,
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

SELECT s.SupplierID, s.SupplierName, SUM(od.Quantity * od.Price) AS TotalRevenue
FROM Suppliers s
JOIN Orders o ON s.SupplierID = o.SupplierID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY s.SupplierID, s.SupplierName;
```

Design a database schema for a University Examination System and generate SQL queries using AI.

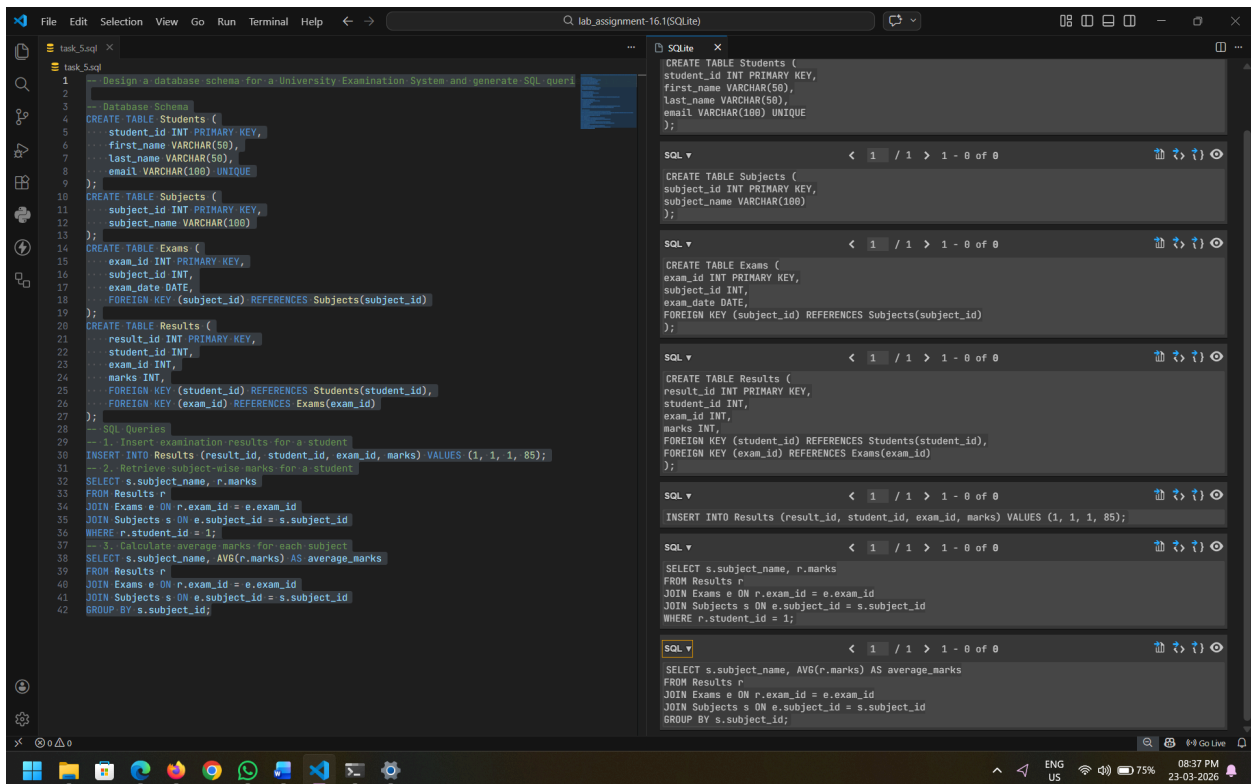
- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:
 - o Insert examination results for a student.
 - o Retrieve subject-wise marks for a student.
 - o Calculate average marks for each subject.

- Normalized schema and SQL queries involving aggregation and joins.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_5.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_5.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:



The screenshot shows a code editor with two panels. The left panel displays a file named 'task_5.sql' containing a database schema and SQL queries. The right panel shows the execution results of these queries in the SQLite console.

task_5.sql

```
1  -- Design a database schema for a University Examination System and generate SQL queries
2
3  Database Schema
4  CREATE TABLE Students (
5      student_id INT PRIMARY KEY,
6      first_name VARCHAR(50),
7      last_name VARCHAR(50),
8      email VARCHAR(100) UNIQUE
9  );
10 CREATE TABLE Subjects (
11     subject_id INT PRIMARY KEY,
12     subject_name VARCHAR(100)
13 );
14 CREATE TABLE Exams (
15     exam_id INT PRIMARY KEY,
16     subject_id INT,
17     exam_date DATE,
18     FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
19 );
20 CREATE TABLE Results (
21     result_id INT PRIMARY KEY,
22     student_id INT,
23     exam_id INT,
24     marks INT,
25     FOREIGN KEY (student_id) REFERENCES Students(student_id),
26     FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
27 );
28 -- SQL Queries
29 -- 1. Insert examination results for a student
30 INSERT INTO Results (result_id, student_id, exam_id, marks) VALUES (1, 1, 1, 85);
31 -- 2. Retrieve subject-wise marks for a student
32 SELECT s.subject_name, r.marks
33 FROM Results r
34 JOIN Exams e ON r.exam_id = e.exam_id
35 JOIN Subjects s ON e.subject_id = s.subject_id
36 WHERE r.student_id = 1;
37 -- 3. Calculate average marks for each subject
38 SELECT s.subject_name, AVG(r.marks) AS average_marks
39 FROM Results r
40 JOIN Exams e ON r.exam_id = e.exam_id
41 JOIN Subjects s ON e.subject_id = s.subject_id
42 GROUP BY s.subject_id;
```

SQLite Console

```
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  email VARCHAR(100) UNIQUE
);

CREATE TABLE Subjects (
  subject_id INT PRIMARY KEY,
  subject_name VARCHAR(100)
);

CREATE TABLE Exams (
  exam_id INT PRIMARY KEY,
  subject_id INT,
  exam_date DATE,
  FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);

CREATE TABLE Results (
  result_id INT PRIMARY KEY,
  student_id INT,
  exam_id INT,
  marks INT,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);

INSERT INTO Results (result_id, student_id, exam_id, marks) VALUES (1, 1, 1, 85);

SELECT s.subject_name, r.marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = 1;

SELECT s.subject_name, AVG(r.marks) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_id;
```